CSE 232
Computer Networks
Assignment-2

Group 15

Anushk Kumar	2023115
Abhas Gupta	2023017
Arhan Jain	2023118
Ayush Kitnawat	2023160
Akshat Singh	2023064

1. Briefly explain your design and the key changes in the e1000 main.c.

Design Implementation

In this assignment, we had to simulate the conversion of a UDP-type Packet to an ICMP packet, and for the ECHO response, we had to again convert the ICMP packet to a UDP-type packet before sending it to the receive socket buffer.

So we have implemented the conversion logic inside two subroutines of the e1000 driver named **e1000_xmit_frame** and **e1000_receive_skb.** The detailed explanation of each of the implementations is given below:

- **e1000_xmit_frame:**
  This subroutine was responsible for performing the send packet operation in the NIC. We have implemented a struct of i*cmphdr* above the routine to typecast a UDP packet to an ICMP packet.

  Along with, we have created a helper function *udp_header_load(struct udphdr * udp_header)* which helps in assigning Magic Number (OxDECAF) in chunks of individual num variables. *checksum*() function to calculate the checksum across different headers.

The routine starts with accessing the IP header from skb, then it checks the protocol parameter; if it is set to **17**, then it's a UDP packet. Then we check if address being sent to is 100.100.100.100 just like in the PA1, then the *udp_header_loader* function is called to extract the Magic Number and convert the provided IP address to target_ip.

The conversion starts by accessing *udphdr* from skb, then changing the protocol id to that of ICMP (1). Then we update the destination address in the ip header to the address received from *udp_loader* and store the source port of the udp header (to fill in icmpheader id).
Then we initialised the checksum for the IP Header to be 0, then updated it with the checksum returned by the checksum calculator function (which we took from lecture slides) mentioned above.

Now a struct of *icmphdr* is created by typecasting the above udpheader struct, we overwrite it onto the udpheader address as both are exactly the same in size. Then *icmphdr* is updated with values as :

```
icmpheader->type = 8;
icmpheader->code = 0;
icmpheader->id = sourceport; //The sourceport we got from
the udpheader
icmpheader->sequence = htons(0);
```

Then, checksum values are also calculated through the same checksum function and updated to the header.
Then we have returned the status of IP_summed to be CHECKSUM_COMPLETE, as directed in the assignment.

- **e1000_receive_skb:**
  This subroutine is responsible for receiving packets on the socket buffer. The changes we have made are by defining a struct of *pseudoheader* for computation of udp header's checksum.

  Ip header is accessed on address skb->data + 14 (as size of ethernet header is 14). It is mentioned in the assignment that skb->data points to ip header but through debug print statements we figured that it points to

ethernet header so we had to skip through that. Then we filter out packets based on the protocol ID, that is, ICMP protocol packets.

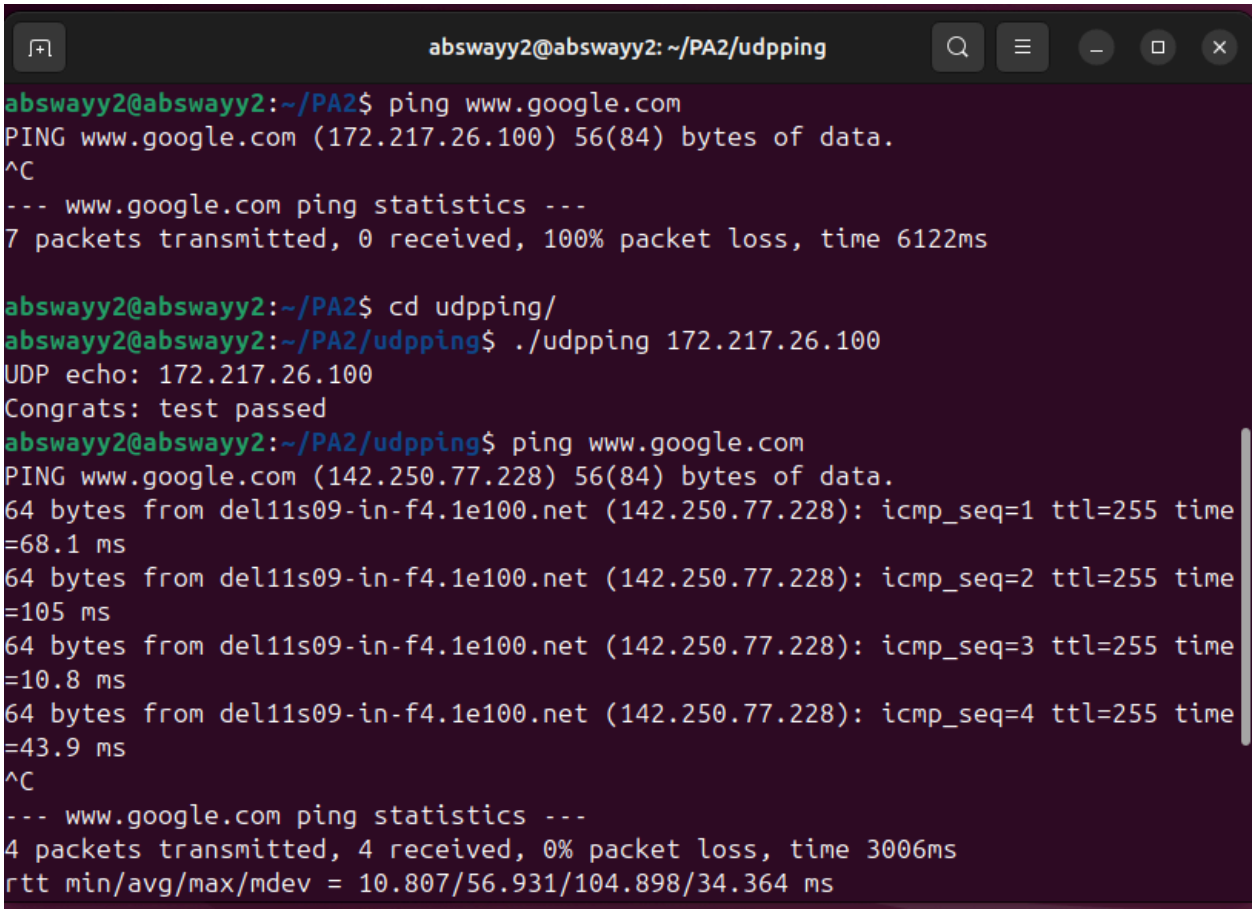
Then, we access the ICMP header, we check for its type, which is supposed to be 0. After that, we check the payload of the ICMP packet if it ends with the payload, we do this by checking if payload is the same as we set the magicnumber variables to in udp_header_loader earlier.

The ICMP header is then typecasted to a UDP header packet for overwriting, where we update values of udpheader, like taking sourceport from icmpheader->id to set udpheader's destination port to it, we also set the pseudoheader's values here. Then we define a char buffer array (size = 1+12 {size of pseudoheader} + 16 {size of udp packet i.e. 8 + 8}) and populate it with entries from the *Pseudoheader* and *udpheader. After* that, we calculate the checksum through this buffer for the UDP header using the same checksum function. After changing networkheader protocol, checksum is calculated and updated the same way as in the transmit function. Then all the information is passed through the default checks and passed to the Kernel.

2. Are you getting the expected output after pinging Google using udpping?
Yes after finding ip of google.com, we are getting Congrats: test passed after executing the ping command to Google using udpping.

```
abswayy2@abswayy2:~/PA2$ cd udpping/
abswayy2@abswayy2:~/PA2/udpping$ ./udpping 172.217.26.100
UDP echo: 172.217.26.100
Congrats: test passed
```

3. Is ping to Google using the ping application work correctly after a successful ping to Google using udpping?
Yes, it's working successfully as seen in the screenshot below.



4. Write the names and roll numbers of all group members.

Anushk Kumar	2023115
Abhas Gupta	2023017
Arhan Jain	2023118
Ayush Kitnawat	2023160
Akshat Singh	2023064